TP 2025/2026 – Extension d'un Système de Messagerie Distribuée

Contexte du travail

Ce TP s'inscrit dans la continuité du travail réalisé par les étudiants de l'année précédente, qui ont développé un système de messagerie distribué en Java intégrant les protocoles SMTP et POP3, un serveur d'authentification RMI, une base de données MySQL et une API REST.

Pour cette année, le système existant sera utilisé comme **socle de base**. Le code vous sera fourni. Votre travail ne consiste pas à redévelopper le système, mais à :

- Comprendre son architecture
- Le déployer et le tester
- Étendre ses fonctionnalités
- Ajouter de nouveaux protocoles et services distribués
- Améliorer sa scalabilité et son accessibilité

L'objectif est de vous placer dans une situation réaliste d'ingénierie : **prise en main d'un système existant et évolution de celui-ci**.

Remarque 1 : Chaque étape du développement doit être versionnée et documentée dans un dépôt Git. Vous devez donc installer et configurer un client Git pour committer (commit) et pousser (push) vos modifications après chaque fonctionnalité ou correction.

Remarque 2 : Un rapport doit être rédigé en plusieurs sections, correspondant aux différentes parties du TP. Après chaque phase de développement et de test, vous devez commenter les développements effectués ainsi que les résultats obtenus.

Objectifs pédagogiques

1. Comprendre une architecture distribuée existante
2. Intégrer un nouveau protocole standard (IMAP)
3. Ajouter une couche Web au système de messagerie
4. Développer des clients applicatifs utilisant des bibliothèques standards
5. Mettre en place un mécanisme de load balancing

1 Prise en main du système existant

Cette première étape a pour objectif de permettre une compréhension globale du système de messagerie fourni. Avant toute extension ou modification, il est nécessaire d'installer, de déployer et de tester l'ensemble des composants afin de valider leur bon fonctionnement, d'identifier les éventuels problèmes et de **proposer des des améliorations**.

Il faut analyser l'architecture existante, vérifier la chaîne complète de traitement d'un message et documenter le comportement du système.

Le PDF complet du TP de l'année précédente est également fourni comme référence afin d'aider à la compréhension de l'architecture globale et des choix d'implémentation.

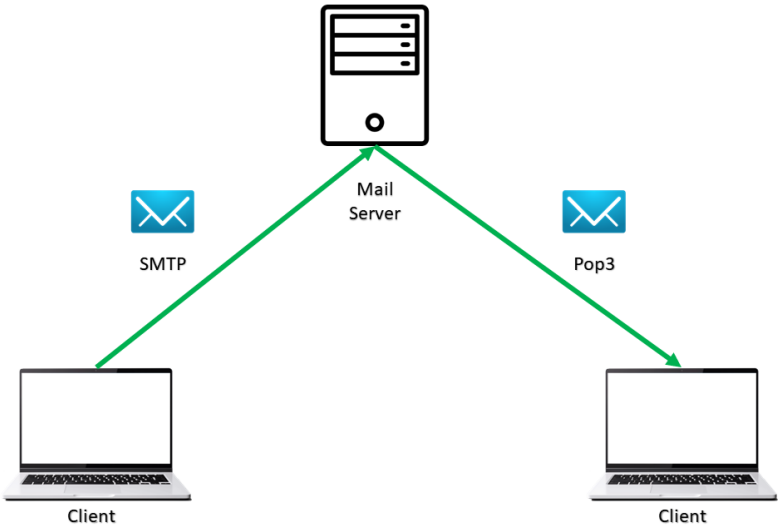

FIGURE 1 – Architecture globale du serveur de messagerie

Objectif

Prendre en main la plateforme existante afin de :

- Comprendre l'architecture logicielle
- Identifier les rôles de chaque composant
- Vérifier le fonctionnement global et améliorer le code

Travail demandé

- Installer et exécuter le système fourni.
- Configurer l'environnement d'exécution (ports et répertoires).
- Déployer et démarrer les composants suivants :
  - Serveur SMTP
  - Serveur POP3
- Vérifier que chaque service démarre correctement.
- Tester la communication entre composants.
- Réaliser des tests fonctionnels complets.

Points d'analyse attendus

Les éléments suivants doivent être identifiés et expliqués :

- Ports utilisés par chaque service
- Protocoles employés (SMTP, POP3)
- Séquence d'authentification utilisateur
- Cycle de vie d'un message (envoi → stockage → récupération)

Tests fonctionnels avec Différents Scénarios

Exécuter plusieurs scénarios afin de valider le comportement du système complet.

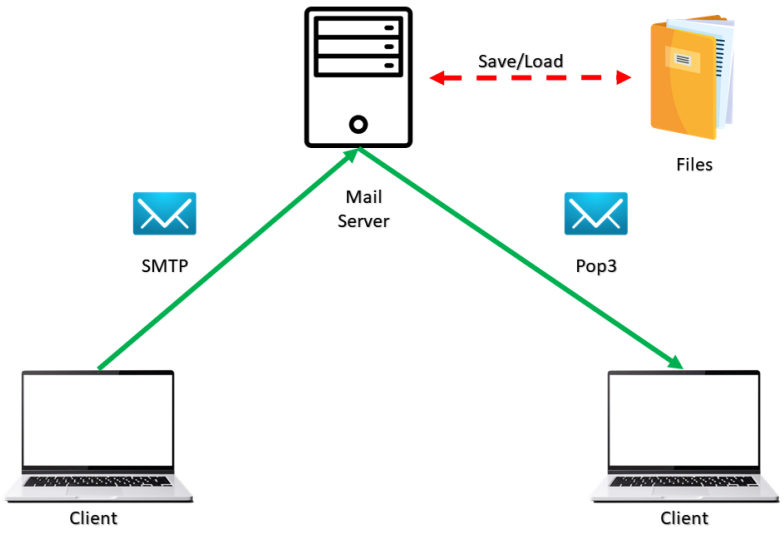

FIGURE 2 – Implémentation des Protocoles SMTP et POP3 avec des Sockets

1.1 Scénarios de test à réaliser

1. **Scénario d'envoi complet :**
   — Envoyer un message via SMTP.
   — Vérifier son stockage.
   — Vérifier sa disponibilité via POP3.
2. **Scénario d'authentification :**
   — Tester une authentification valide.
   — Tester une authentification invalide.
   — Observer les réponses retournées.
3. **Scénario erreurs :**
   — Arrêter un composant.
   — Observer l'impact sur le système.
   — Identifier les dépendances critiques.

Outils recommandés

— Telnet / Netcat (tests protocoles)
— Clients mail (Thunderbird) ou **Outlook**
— Outils de logs

1.2 Discussion

— Analysez les deux protocoles SMTP et POP3.
— Identifiez les points forts et limitations.
— Proposez des améliorations potentielles.

2 Développement du serveur IMAP :

Le serveur IMAP qui implémente le protocole IMAP doit permettre l'accès distant aux boîtes aux lettres tout en conservant les messages sur le serveur. Contrairement à POP3, IMAP supporte la gestion des dossiers, la consultation partielle des messages et la gestion des états (lu, non lu, marqué, etc.).

Votre implémentation doit gérer les commandes IMAP de base (**LOGIN, SELECT, FETCH, STORE, SEARCH, LOGOUT**) et répondre avec les codes et formats de réponses appropriés conformément à la RFC 9051 [1].

Téléchargement et lecture de la RFC

Veuillez vous référer à la spécification du protocole IMAP : **RFC 9051** afin de comprendre la structure des commandes, le format des réponses et le modèle de session.

Respect des normes du protocole via un automate à états finis

Comme pour SMTP et POP3, le protocole IMAP suit un modèle à états (Non authentifié, Authentifié, Boîte sélectionnée, Logout). Vous devez implémenter un automate à états finis afin de contrôler les commandes autorisées dans chaque état.

Par exemple :

— La commande `LOGIN` n'est autorisée que dans l'état non authentifié.

— La commande `SELECT` n'est autorisée qu'après authentification.

— Les commandes `FETCH` et `STORE` ne sont autorisées que lorsqu'une boîte est sélectionnée.

Toute commande invalide dans un état donné doit être rejetée avec une réponse d'erreur appropriée.

Organisation des boîtes aux lettres

Le serveur doit supporter une organisation logique des messages par dossiers (par exemple : **INBOX, SENT, TRASH**). Ces dossiers peuvent être mappés soit vers la base de données existante, soit vers une structure logique dérivée.

Chaque message doit posséder au minimum :

— un identifiant unique

— un état (lu / non lu)

— des indicateurs (flags)

Dans cette étape, pour des raisons de simplicité, vous pouvez limiter l'implémentation à un seul dossier **INBOX**. Les autres dossiers peuvent être simulés ou optionnels. Les flags peuvent être limités à `\Seen` (lu / non lu).

2.1 Tests du Protocole IMAP avec Différents Scénarios

Valider le bon fonctionnement du serveur IMAP en testant différents scénarios d'interaction. Ces tests permettront de vérifier que le serveur respecte les spécifications du protocole IMAP et gère correctement les états de session, les commandes et les réponses structurées.

Scénarios de test à implémenter

1. **Scénario de base :**
   — Connexion au serveur IMAP.
   — Authentification avec la commande `LOGIN`.
   — Sélection de la boîte `INBOX` avec `SELECT`.
   — Lecture d'un message avec `FETCH`.
   — Déconnexion avec `LOGOUT`.
   — Vérifier la conformité du format des réponses.
2. **Scénario avec sélection de boîte inexistante :**
   — Tenter `SELECT UNKNOWN`.
   — Vérifier que le serveur renvoie une réponse d'erreur appropriée.
3. **Scénario de lecture partielle :**
   — Utiliser `FETCH` pour récupérer uniquement les en-têtes.

1. https ://www.ietf.org/rfc/rfc9051.html

— Vérifier que le serveur ne retourne pas le corps complet du message.

4. **Scénario de gestion des flags :**
   — Marquer un message comme lu via `STORE`.
   — Vérifier que l'état est conservé lors d'une nouvelle session.
5. **Scénario de recherche :**
   — Utiliser la commande `SEARCH` par expéditeur ou sujet.
   — Vérifier que seuls les messages correspondants sont retournés.
6. **Scénario avec commandes dans le mauvais état :**
   — Exécuter `FETCH` avant `LOGIN` ou `SELECT`.
   — Vérifier que la commande est rejetée.

Outils de test

— Utilisez **Telnet** pour tester manuellement les commandes IMAP.
— Vous pouvez également utiliser des clients compatibles IMAP comme **Thunderbird** ou **Outlook**.
— Activez les logs serveur pour analyser les échanges client–serveur.

2.2 Discussion

— Pour chaque scénario, documentez les réponses du serveur et vérifiez la conformité avec la RFC IMAP.
— Fournissez des captures d'écran ou des logs des échanges.
— Analysez les différences observées avec POP3 (gestion des états, flags, dossiers).
— Discutez les limites de votre implémentation et les extensions possibles.

3 Interfaces de Supervision pour la Distribution des Serveurs de Messagerie

Important : Avant de commencer la partie suivante sur le serveur d'authentification avec Java RMI, vous devez d'abord développer des interfaces graphiques permettant de superviser et contrôler les serveurs SMTP, POP3 et IMAP que vous avez implémentés.

Maintenant que les protocoles SMTP, POP3 et IMAP sont correctement implémentés et fonctionnels, l'étape suivante consiste à améliorer l'outil de test et d'administration du système en développant des interfaces graphiques pour les serveurs.

L'objectif est de faciliter l'exploitation et l'observation du comportement des serveurs tout en préparant une architecture permettant la distribution des différents composants du système.

3.1 Principe général

Vous devez développer une interface graphique indépendante pour chacun des serveurs suivants :

— Serveur SMTP
— Serveur POP3
— Serveur IMAP

Chaque interface doit permettre de contrôler et surveiller le serveur correspondant. Ces interfaces doivent être conçues de manière indépendante afin de permettre, dans une étape ultérieure, de déployer chaque serveur sur une machine différente dans un environnement distribué.

3.2 Fonctionnalités minimales

Chaque interface graphique doit permettre les fonctionnalités suivantes :

— **Démarrer le serveur** : lancer le serveur et ouvrir le port réseau correspondant.

— **Arrêter le serveur** : arrêter proprement le serveur et libérer les ressources utilisées.

— **Afficher l'historique des commandes** : visualiser en temps réel les commandes envoyées par les clients et les réponses du serveur.

L'historique doit permettre d'observer clairement les interactions entre le client et le serveur, par exemple :

```
Client1 -> HELO eoc.dz
Serveur -> 250 Hello

Client1 -> MAIL FROM:mohammed@eoc.dz
Serveur -> 250 OK

Client2 -> USER Karim
Serveur -> +OK
```

3.3 Fonctionnalités recommandées (bonus)

Pour améliorer l'outil d'administration, vous pouvez ajouter les fonctionnalités suivantes :

— affichage du nombre de clients connectés.

— journalisation des événements (connexion, déconnexion, erreurs).

— horodatage des commandes reçues.

4 Serveur d'Authentification avec Java RMI

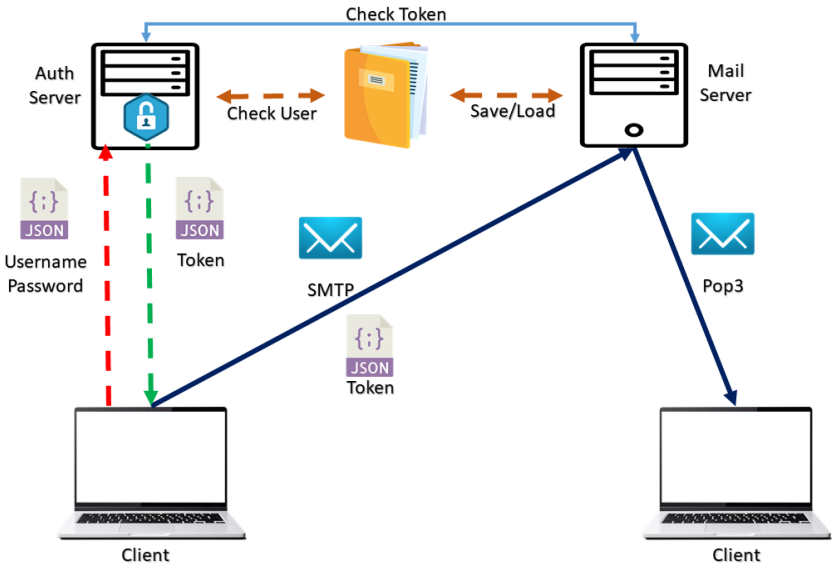

FIGURE 3 – Serveur d'Authentification avec Java RMI

Dans cette partie, vous devez implémenter un **serveur d'authentification distribué** utilisant **Java RMI**. Ce serveur permettra de gérer les comptes utilisateurs et sera utilisé par les serveurs **SMTP**, **POP3** et **IMAP** pour vérifier l'identité des utilisateurs.

4.1 Serveur d'authentification RMI

Implémentez un service Java RMI offrant les opérations suivantes :

— Vérification des identifiants utilisateur (login / mot de passe).

— Création d'un nouveau compte utilisateur.

— Mise à jour ou suppression d'un compte existant.

Les comptes utilisateurs doivent être stockés localement dans un fichier **JSON ou XML**. Le serveur RMI doit lire et mettre à jour ce fichier afin de conserver les données de manière persistante.

4.2 Intégration avec SMTP, POP3 et IMAP

Les serveurs **SMTP**, **POP3** et **IMAP** doivent utiliser le serveur RMI pour l'authentification des utilisateurs :

— Lors d'une connexion POP3 ou IMAP, le serveur vérifie l'identifiant et le mot de passe via le serveur RMI.

— Avant l'envoi d'un email, le serveur SMTP vérifie que l'expéditeur est un utilisateur valide.

4.3 Client d'administration RMI

Développez un **client Java RMI** permettant de gérer les comptes utilisateurs. Ce client doit être une application avec interface graphique simple et doit permettre :

— d'ajouter un utilisateur,

— de modifier un utilisateur,

— de supprimer un utilisateur.

4.4 Tests et validation

Vérifiez le bon fonctionnement du système avec les scénarios suivants :

— Création d'un compte utilisateur via le client RMI.

— Suppression d'un compte et vérification qu'il n'est plus accepté par SMTP/POP3/IMAP.

— Authentification avec identifiants valides.

— Authentification avec identifiants invalides.

— Vérification que le serveur SMTP consulte correctement le serveur RMI.

Dans votre rapport, documentez les tests réalisés (logs ou captures d'écran) et discutez les problèmes rencontrés ainsi que les améliorations possibles.

5 Intégration d'une Base de Données MySQL pour la Distribution des Emails

Pour améliorer la gestion des emails et des utilisateurs, le système passe d'un stockage local (fichiers) à une base de données MySQL centralisée. Cette approche permet :

— Une **gestion centralisée** des comptes et messages.

— Une **cohérence et sécurité des données** renforcées.

— Une **scalabilité** adaptée à un grand volume d'emails et à une architecture distribuée.

Désormais :

— Le serveur **SMTP** stocke les emails en base.

— Les serveurs **POP3** et **IMAP** récupèrent les emails depuis MySQL.

— Le serveur d'authentification gère les comptes utilisateurs directement en base.

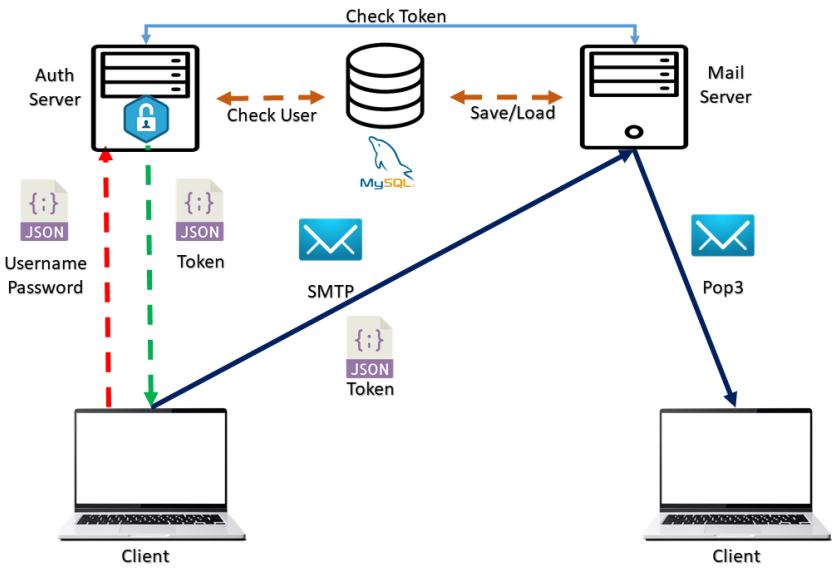

FIGURE 4 – Architecture distribuée avec base de données MySQL

5.1 Conception de la Base de Données

Deux tables principales suffisent pour le fonctionnement minimal :

— **users** : informations des utilisateurs (identifiant, mot de passe haché, date de création, statut, etc.).

— **emails** : messages stockés avec métadonnées (expéditeur, destinataire, sujet, date, état de lecture, contenu).

Le schéma complet doit être présenté dans le rapport sous forme d'un diagramme UML (Modèle Entité-Association) avec justification des choix de conception.

5.2 Procédures Stockées

Pour optimiser les interactions, les serveurs utilisent des procédures stockées, par exemple :

— `authenticate_user(username, password)` : vérification des identifiants.

— `store_email(sender, recipient, subject, content)` : insertion d'un email.

— `fetch_emails(user)` : récupération des emails pour un utilisateur.

— `delete_email(email_id)` : suppression d'un email.

— `update_password(user, new_password)` : modification du mot de passe.

5.3 Intégration avec les Serveurs

Chaque serveur est adapté pour communiquer avec MySQL via JDBC :

— **SMTP** : envoie d'emails via `store_email()`.

— **POP3 et IMAP** : récupération et suppression d'emails via `fetch_emails()` et `delete_email()`. IMAP offre la synchronisation en ligne des messages.

— **Serveur d'authentification** : gestion des comptes utilisateurs avec `authenticate_user()`.

5.4 Tests et Validation

Vérifications principales :

— **Fonctionnels** : création/utilisateurs, envoi, récupération et suppression d'emails via SMTP, POP3 et IMAP.

— **Performance** : temps d'accès, charge avec utilisateurs simultanés.

— **Robustesse et sécurité** : tentatives de connexion invalides, protection contre injections SQL.

5.5 Discussion et Analyse

- Documenter le comportement des serveurs pour vérifier la conformité aux protocoles.
- Fournir captures d'écran ou logs pour valider les tests.
- Analyser les résultats, noter les problèmes rencontrés et proposer des améliorations.

6 Intégration d'une Interface Web de Messagerie

L'objectif de cette partie est d'ajouter une interface Web au système de messagerie existant afin de permettre l'accès aux fonctionnalités principales (consultation et envoi d'emails) à travers un navigateur Web. Cette interface constitue une couche supplémentaire au-dessus des services distribués déjà développés (API REST, SMTP, POP3, IMAP, authentification).

Cette approche permet d'illustrer la séparation des couches, l'interopérabilité des services et l'exposition des fonctionnalités distribuées à travers une interface utilisateur Web.

Architecture de la solution

L'interface Web doit s'appuyer sur l'architecture distribuée existante et ne doit pas accéder directement à la base de données. Toutes les opérations doivent passer par les services exposés (API REST ou serveurs de messagerie).

L'architecture doit comporter :

- Un **serveur Web** (Apache, Nginx ou serveur intégré)
- Un **serveur applicatif** (PHP, Java Web ou framework équivalent)
- Une **communication avec l'API REST existante** via HTTP/JSON

Le serveur Web agit comme client des services distribués déjà implémentés.

Il est interdit dans cette étape d'accéder directement à la base de données depuis l'interface Web. Toutes les opérations doivent passer par l'API REST ou les services serveurs existants.

Fonctionnalités à implémenter

L'interface Web doit permettre au minimum les opérations suivantes :

- Authentification utilisateur (login / logout)
- Consultation de la boîte de réception
- Lecture d'un message
- Suppression d'un message
- Envoi d'un nouveau message

Chaque action Web doit correspondre à un appel vers un service distant (REST, SMTP ou IMAP selon votre architecture).

Gestion des sessions Web

L'authentification utilisateur doit être maintenue via un mécanisme de session :

- Session HTTP côté serveur
- Ou token JWT fourni par l'API REST

Les pages de consultation des messages ne doivent être accessibles qu'après authentification valide.

Flux de communication

Les flux suivants doivent être respectés :

- Login Web → API REST Authentification
- Consultation inbox → API REST récupération emails
- Envoi message → API REST ou SMTP
- Suppression → API REST suppression message

Dans le rapport, vous devez fournir un diagramme de séquence illustrant au moins un scénario complet d'envoi et de consultation d'email via l'interface Web.

6.1 Tests de l'Interface Web avec Différents Scénarios

Valider le bon fonctionnement de l'interface Web en testant différents scénarios d'utilisation. Ces tests doivent vérifier la bonne intégration avec les services distribués existants.

Scénarios de test à implémenter

1. **Scénario de connexion valide :**
   - Authentification avec identifiants corrects.
   - Accès à la boîte de réception.
   - Vérifier la création de la session.
2. **Scénario de connexion invalide :**
   - Tentative de connexion avec mot de passe incorrect.
   - Vérifier le refus d'accès et le message d'erreur.
3. **Scénario de consultation inbox :**
   - Affichage de la liste des messages.
   - Vérifier la cohérence avec les données en base.
4. **Scénario de lecture message :**
   - Ouverture d'un message.
   - Vérifier l'affichage complet du contenu.
   - Vérifier la mise à jour de l'état lu / non lu si supporté.
5. **Scénario d'envoi message :**
   - Envoi d'un email via le formulaire Web.
   - Vérifier la réception côté destinataire.
   - Vérifier le passage par les services serveurs.
6. **Scénario de suppression :**
   - Suppression d'un message.
   - Vérifier la disparition dans l'interface.
   - Vérifier la suppression effective côté serveur.
7. **Scénario d'accès non autorisé :**
   - Accès direct à l'URL inbox sans authentification.
   - Vérifier la redirection vers la page login.

Outils de test

- Navigateur Web (tests fonctionnels)
- Outils de développement navigateur (logs réseau HTTP)
- Postman ou cURL pour vérifier les appels API
- Logs serveur Web et serveur REST

6.2 Discussion

- Documentez les échanges entre l'interface Web et les services distribués.
- Fournissez des captures d'écran des différents scénarios.
- Analysez la séparation entre couche présentation et couche services.
- Discutez les avantages et limites d'une interface Web reposant sur des services distribués.
- Proposez des améliorations possibles (pagination, recherche, pièces jointes, etc.).

7 Développement de clients de messagerie

Dans cette phase, il ne s'agit plus d'implémenter les protocoles côté serveur, mais de développer des **clients de messagerie** capables d'interagir avec les serveurs SMTP, POP3 et IMAP déjà mis en place. Les clients doivent s'appuyer sur des bibliothèques standards afin d'éviter la réimplémentation bas niveau des protocoles.

L'objectif est de valider l'interopérabilité complète du système de messagerie à travers des applications clientes.

Objectif

Développer des clients Java permettant :

— L'envoi d'emails via SMTP

— La lecture des messages via POP3

— La consultation avancée via IMAP

— La gestion des opérations de base sur les messages

Technologies à utiliser

Les clients doivent être développés en Java en utilisant des bibliothèques standards telles que :

— **JavaMail API** (Jakarta Mail)

Il est strictement interdit de réimplémenter les protocoles SMTP, POP3 ou IMAP à bas niveau dans cette phase. L'objectif est l'utilisation correcte d'une bibliothèque existante.

Clients à développer

Trois clients doivent être réalisés :

— **Client SMTP** — envoi de messages

— **Client POP3** — récupération simple des messages

— **Client IMAP** — gestion avancée de la boîte mail

Chaque client peut être :

— en ligne de commande

— ou avec interface graphique simple

Fonctionnalités minimales

Les fonctionnalités suivantes doivent être implémentées :

— Authentification utilisateur

— Envoi de message

— Lecture de la boîte de réception

— Suppression de message

— Recherche de message (IMAP)

— Affichage des en-têtes et du contenu

Contraintes d'implémentation

— Paramétrage configurable (serveur, port, utilisateur, mot de passe)

— Gestion des erreurs de connexion

— Gestion des exceptions réseau

— Affichage clair des messages d'erreur

Pour simplifier, l'authentification peut être réalisée par login/mot de passe sans chiffrement TLS dans un premier temps.

7.1 Tests des clients avec Différents Scénarios

Valider le bon fonctionnement des clients développés en exécutant plusieurs scénarios d'interaction avec vos serveurs de messagerie.

Scénarios de test à implémenter

1. **Scénario d'envoi SMTP :**
   - Envoyer un email vers un utilisateur local.
   - Vérifier la réception via POP3 et IMAP.
   - Vérifier les en-têtes du message.
2. **Scénario lecture POP3 :**
   - Lister les messages.
   - Lire un message.
   - Supprimer un message.
   - Vérifier la suppression après reconnexion.
3. **Scénario lecture IMAP :**
   - Ouvrir INBOX.
   - Lire plusieurs messages.
   - Marquer comme lu / non lu.
   - Effectuer une recherche par sujet.
4. **Scénario erreurs d'authentification :**
   - Utiliser un mot de passe incorrect.
   - Vérifier la gestion d'erreur.
5. **Scénario serveur indisponible :**
   - Arrêter le serveur.
   - Lancer le client.
   - Vérifier la gestion des exceptions.

Outils de validation

- JavaMail API
- Logs serveur
- Clients de référence (Thunderbird) pour comparaison

7.2 Discussion

- Comparez le comportement POP3 vs IMAP.
- Analysez les différences de fonctionnalités.
- Discutez la facilité d'intégration via JavaMail.
- Identifiez les limitations rencontrées.
- Proposez des extensions possibles (pièces jointes, HTML, TLS).

8 Scalabilité et Load Balancing

Dans un système de messagerie distribué, la montée en charge et la disponibilité des services sont des aspects essentiels. Lorsqu'un grand nombre de clients accèdent simultanément aux services (API REST, IMAP, POP3, interface Web), un seul serveur peut devenir un point de saturation ou de défaillance.

L'objectif de cette partie est d'introduire un mécanisme de **répartition de charge (Load Balancing)** en dupliquant les serveurs de messagerie et en plaçant un répartiteur de charge en frontal. Cette architecture permet d'améliorer la scalabilité et la tolérance aux pannes du système.

Objectif

Mettre en place une architecture scalable basée sur :

- Plusieurs instances du serveur de messagerie (API / services)
- Un **load balancer** (NGINX ou HAProxy)
- Une stratégie de répartition de charge (Round Robin)
- Des tests de tolérance aux pannes

Principe d'architecture

Le load balancer agit comme un **reverse proxy** placé devant plusieurs instances serveurs. Les clients n'accèdent plus directement aux serveurs applicatifs mais passent par le répartiteur de charge qui distribue les requêtes.

Chaque instance serveur doit être déployée sur :

- un port différent
- ou une machine virtuelle différente
- ou un conteneur différent

Toutes les instances doivent accéder à la même base de données afin de garantir la cohérence des messages.

Pour simplifier le déploiement, vous pouvez exécuter plusieurs instances serveur sur la même machine en utilisant des ports différents.

8.1 Mise en place du Load Balancer

Le load balancer doit être configuré avec :

- Une liste des serveurs backend
- Une politique de répartition **Round Robin**
- Un mécanisme de détection de serveur indisponible

Deux solutions sont acceptées :

- **NGINX** comme reverse proxy
- **HAProxy** comme répartiteur de charge

Déploiement avec NGINX

NGINX doit être configuré pour :

- Recevoir les requêtes clients
- Les transmettre aux instances backend
- Distribuer les requêtes de manière circulaire (Round Robin)

Vous devez fournir dans le rapport le fichier de configuration utilisé ainsi que son explication.

Contraintes de cohérence

Dans une architecture avec plusieurs instances :

- Les sessions utilisateurs doivent rester cohérentes
- Les données doivent être partagées (base commune)
- Les écritures concurrentes doivent rester correctes

Si la gestion de session distribuée est complexe, vous pouvez utiliser des sessions sans état (stateless) via token JWT.

8.2 Tests de Load Balancing avec Différents Scénarios

Valider le bon fonctionnement du mécanisme de répartition de charge en exécutant plusieurs scénarios de test. Ces tests doivent démontrer la distribution effective des requêtes et la continuité de service en cas de panne.

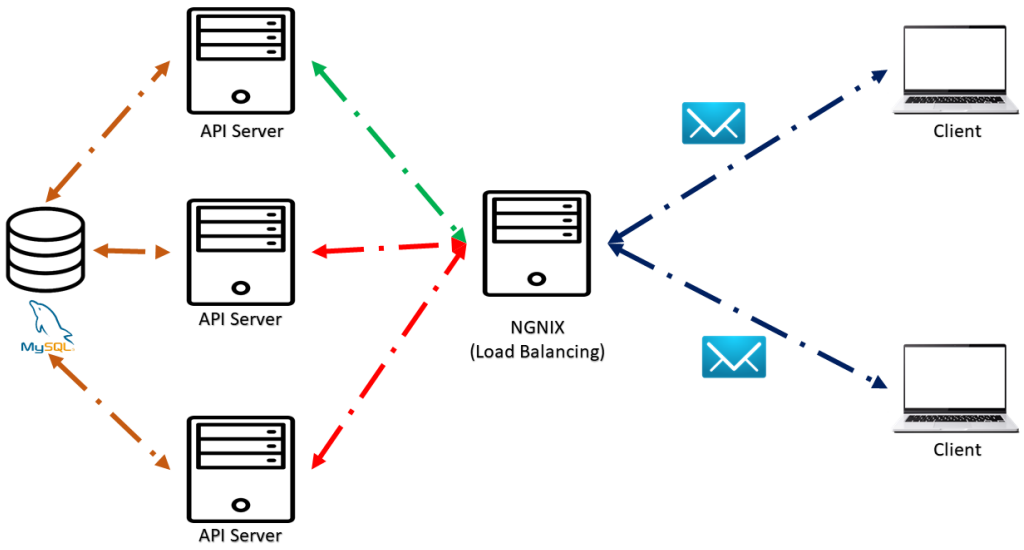

FIGURE 5 – Scalabilité et Load Balancing

Scénarios de test à implémenter

1. **Scénario de répartition simple :**
   — Envoyer plusieurs requêtes successives.
   — Vérifier dans les logs que les requêtes sont distribuées entre les serveurs.
   — Identifier le serveur ayant traité chaque requête.
2. **Scénario de charge concurrente :**
   — Simuler plusieurs clients simultanés.
   — Vérifier la distribution de charge.
   — Mesurer les temps de réponse.
3. **Scénario de saturation :**
   — Augmenter progressivement le nombre de requêtes.
   — Observer le comportement global du système.
   — Comparer avec un déploiement mono-serveur.
4. **Scénario de panne d'un nœud :**
   — Arrêter une instance serveur backend.
   — Continuer les requêtes clients.
   — Vérifier que le load balancer redirige vers les instances restantes.
5. **Scénario de reprise :**
   — Redémarrer le serveur arrêté.
   — Vérifier sa réintégration dans la rotation.

Tests de charge

Des tests de charge doivent être réalisés avec :

— **JMeter**
— **Apache Bench (ab)**

Les mesures suivantes doivent être collectées :

— Temps de réponse moyen
— Nombre de requêtes par seconde
— Taux d'erreurs
— Répartition des requêtes par nœud

Outils de test

- JMeter
- Apache Bench
- Logs NGINX / HAProxy
- Logs serveurs backend

8.3 Discussion

- Présentez l'architecture déployée.
- Analysez la distribution réelle de la charge.
- Comparez les performances avec et sans load balancer.
- Discutez la tolérance aux pannes observée.
- Identifiez les limites de votre configuration.
- Proposez des améliorations (health checks avancés, sticky sessions, auto-scaling).